

AI for Science Applied to Algorithmic Trading: An Agent-Governed, Closed-Loop Discovery Architecture

Alejandro Reynoso

Chief Scientist, DEFI Capital

June 10, 2026

Abstract

This paper develops a closed-loop research architecture for algorithmic equity selection inspired by the *AI for Science* (AI4S) paradigm. Rather than claiming that language models can replace empirical finance, the paper shows how they can be placed inside a governed workflow in which hypotheses are proposed, encoded, tested, stress-tested, and either rejected or retained under explicit criteria. The architecture transforms market observations into cross-sectional features, constrains a large language model (LLM) to generate bounded alpha hypotheses, evaluates those hypotheses through transaction-cost-aware backtesting and adverse-scenario analysis, and delegates continuation decisions to an `EvaluationAgent` operating on typed `CycleResult` records. The design addresses recurring weaknesses in quantitative research, including informal hypothesis tracking, data-snooping risk, brittle backtests, insufficient treatment of transaction costs, and opaque model-driven reasoning. Each cycle preserves performance, fragility, feedback, and next-round search context in an auditable record. The paper specifies the workflow, software interfaces, mathematical core, LLM protocols, stress scenarios, and numerical safeguards required for a reproducible AI-assisted discovery system. It is accompanied by two computational notebooks: one implements the architecture faithfully, and the other extends it with structured-output discipline, additional research loops, model-gate controls, and exportable audit artifacts. The contribution is methodological: a reproducible blueprint for using generative AI in financial hypothesis formation, validation, governance, and review, not an investment recommendation.

Keywords: artificial intelligence for science; algorithmic trading; alpha discovery; backtesting; stress testing; model governance.

1. Introduction

Financial alpha discovery is an empirical research problem conducted under severe epistemic constraints. The data are noisy, nonstationary, adaptive, and subject to multiple-testing bias. A strategy that appears attractive in a historical sample may be the product of luck, data leakage, an unmodeled liquidity premium, unaccounted transaction costs, or a transient market regime. For this reason, a credible research process must distinguish sharply between a plausible economic story, a statistically suggestive result, and a deployable trading rule. This paper contends that the AI4S paradigm is valuable for algorithmic trading precisely because it requires that distinction to be made explicit. A model-generated signal is treated as a hypothesis to be tested, not as an output to be trusted.

AI4S denotes a mode of scientific work in which artificial intelligence participates actively in the research cycle: representing observations, proposing hypotheses, selecting experiments, interpreting failures, and updating the next search direction [1]. This mode has been influential in domains such as protein-structure prediction [2], materials discovery [3], and autonomous chemistry [4]. Finance differs from those laboratory settings in an important respect: markets are reflexive systems in which the act of exploiting a pattern can alter the pattern itself. The analogy therefore cannot be transferred naively. The relevant transfer is procedural: AI4S provides a disciplined grammar for moving from observation to representation, from representation to hypothesis, from hypothesis to test, and from test to revision.

The architecture developed here applies that grammar to equity selection. Raw prices and volumes are transformed into ranked cross-sectional features. An LLM proposes candidate alpha hypotheses under a

constrained feature vocabulary. Those hypotheses are evaluated by a deterministic backtesting engine with transaction costs and weekly rebalancing. The best candidate is then subjected to stress scenarios that perturb volatility, liquidity, and crowding. A separate `EvaluationAgent` reads the resulting `CycleResult` and decides whether the research process should stop, continue with failure context, or halt after exhausting the permitted number of rounds. This division of labor is central: the LLM may generate and summarize, but it does not grade itself, modify the test rules, or silently carry state across cycles.

The design reflects several principles from empirical asset pricing and financial econometrics. First, evidence must be path-aware. A return series is not adequately summarized by average return alone; drawdowns, volatility, and turnover are economically material. Second, validation must be adversarial. A signal that survives only in the base case but fails when liquidity is removed or alpha is crowded should be considered fragile. Third, hypotheses must be specified before evaluation. A generated rule is converted into a bounded buy condition, not left as an interpretive paragraph. Fourth, the research process must preserve negative evidence. Failed hypotheses are valuable if they improve future search; they are dangerous if they are forgotten and regenerated under a new name.

The paper is organized as follows. Section 2 describes the end-to-end workflow and the role of the loop-back gate. Section 3 explains the software architecture and the separation between the inner research cycle and the outer governor. Section 4 specifies the mathematical core used for feature construction, backtesting, performance measurement, and portfolio formation. Section 5 details the agent decision protocol. Section 6 presents the numerical controls that prevent pathological or misleading results. Section 7 formalizes the LLM interface. Section 8 describes the stress scenarios. The appendix documents the two companion computational notebooks and distinguishes the faithful implementation from the governed audit extension. Section 9 discusses contributions, limitations, and future research directions.

The intended contribution is methodological. The system does not claim that synthetic data can establish tradable alpha, nor does it claim that an LLM can substitute for domain expertise. Instead, it demonstrates how generative AI can be placed inside a rigorous research envelope: one that is typed, auditable, scenario-aware, and explicit about failure. In an institutional review setting, the relevant standard is not whether the model can produce a favorable signal on demand. The relevant standard is whether the research process exposes assumptions, penalizes fragility, records decisions, and remains open to independent critique at each interface. That is the standard this architecture is designed to meet.

2. Workflow Overview

The workflow is a closed-loop research procedure rather than a linear signal-generation pipeline. Each cycle begins with market observations, constructs features, generates candidate hypotheses, evaluates them under a common backtesting protocol, and either advances the strongest candidate or returns failure context to the next round of search. The loop is deliberately conservative. It is designed to reduce the probability that a strategy advances only because it produced a favorable statistic in one environment. Table 1 summarizes the ten stages.

The workflow contains two nested loops. The inner loop is implemented by `AI4SLoop`; it performs feature construction, hypothesis generation, backtesting, stress testing, portfolio construction, paper trading, deployment simulation, feedback, and update generation. The outer loop is implemented by `EvaluationAgent`; it determines whether the result of the inner loop justifies stopping, continuing, or halting. This nesting is methodologically material because model-generated ideas and agentic governance perform different functions. The former explores the hypothesis space; the latter evaluates whether continued search is justified by the evidence.

The critical control point is the loop-back gate after stress testing. A candidate can perform well in the base backtest and still be unsuitable for further deployment simulation if it is highly sensitive to volatility

Table 1: Ten-stage AI4S trading workflow.

Stage	Description
1. Raw Data	Synthetic universe: 50 stocks $\times$ 756 days
2. Features	Cross-sectional momentum, volatility, liquidity, and regime proxies
3. Hypotheses	LLM generates bounded, testable alpha hypotheses
4. Backtest	Weekly-rebalanced equal-weight NAV with transaction costs
5. Stress Tests	Volatility spike, liquidity crisis, and crowding decay scenarios
6. Portfolio	Composite alpha score, position caps, and volatility scaling
7. Paper Trading	Short-horizon mark-to-market simulation with slippage
8. Deployment	Controlled deployment simulation with risk target and kill switch
9. Feedback	LLM diagnoses expected versus realized performance gap
10. Update	LLM proposes refined hypotheses and next-cycle search seeds

shocks, liquidity withdrawal, or alpha decay. The gate therefore classifies a cycle as fragile if any stress scenario produces a negative Sharpe ratio or a maximum drawdown worse than -40% . Fragility does not terminate the research process by itself. Instead, it produces structured failure context that conditions the next hypothesis-generation prompt. The design treats failure as information, not as an exception to be discarded.

This workflow also imposes a hierarchy of evidence. A generated hypothesis has low evidentiary status until it is encoded in a deterministic rule. A base backtest improves that status only modestly, because it may still be overfit or regime-specific. Stress testing improves the evidence by asking whether the hypothesis survives adverse perturbations. Paper-trading and deployment simulations further examine implementation realism. Finally, the agent decision log records why the process accepted, rejected, or revised the candidate. The hierarchy is intended to prevent a common error in quantitative research: promoting an idea directly from narrative plausibility to deployment without adequate adversarial testing.

3. Software Architecture

The implementation is organized around three objects: a cycle-result record, an inner-loop engine, and an outer evaluation agent. This separation is not simply an implementation preference; it is a governance requirement. A research system that mixes hypothesis generation, performance evaluation, state accumulation, and deployment decisions in a single object is difficult to audit. By contrast, typed records and explicit ownership boundaries allow a reviewer to reconstruct what happened in each cycle and why the outer loop made its subsequent decision.

3.1. CycleResult Dataclass

`CycleResult` is an immutable dataclass that stores the complete output of one research cycle. Its role is analogous to a research audit record: it preserves what was proposed, what was tested, what failed, and what information is available for the next experiment. Key fields include generated hypotheses, backtest results, base statistics, stress-test tables, fragility status, paper and deployment-simulation returns, kill-switch status, feedback verdict, and next-cycle hypothesis seeds. The purpose is to prevent the research process from relying on informal memory or unstructured logs.

The dataclass is especially important in an LLM-mediated system because model outputs can be semantically rich but structurally unstable. A language model may describe similar ideas with different words across cycles. The dataclass therefore maps those ideas into stable fields before they influence downstream logic. The `EvaluationAgent` reads an object with known attributes rather than parsing unstructured text. This

reduces operational ambiguity and supports reproducibility: a future reviewer can inspect the same fields and recover the same decision context.

3.2. AI4SLoop: Inner Research Cycle

AI4SLoop executes the inner research procedure. It is instantiated with a cycle number and optional failure context supplied by the outer agent. Its methods build features, generate hypotheses, backtest all candidates, apply stress tests, construct a portfolio, simulate paper trading, simulate controlled deployment, obtain feedback, and generate next-round updates. The class is deliberately stateless across cycles. Persistent knowledge is not stored implicitly inside the object; it is passed explicitly as failure context and returned explicitly through `CycleResult`.

The execution order is auditable by design:

- `build_features()`
- `generate_hypotheses()`
- `backtest_all()`
- `stress_test()`
- `build_portfolio()`
- `paper_trade()`
- `deploy_simulation()`
- `get_feedback()`
- `generate_updates()`

This ordering prevents the LLM from retrofitting hypotheses after observing performance and prevents deployment logic from being evaluated before fragility has been assessed.

3.3. EvaluationAgent: Outer Governor

EvaluationAgent owns the multi-round state of the research program. It stores accumulated failure context, rejected strategy names, and a decision log. Its role is to prevent the inner loop from becoming an uncontrolled search process. After each cycle, the agent performs a deterministic prescreen and, when necessary, requests a structured LLM decision. The output is not free-form commentary; it is a decision object with a reason and a failure summary.

The governor is also the mechanism by which negative evidence becomes reusable. If a strategy fails under liquidity withdrawal, the next prompt can explicitly instruct the model not to regenerate the same high-volatility exposure. If a strategy is stopped after success, the decision log records which evidence was sufficient. If the maximum round limit is reached, the system halts without continuing an indefinite search. In this sense, the outer agent functions as a research-control layer rather than a trading signal generator.

4. Mathematical Core

The mathematical core defines the quantities over which the system is allowed to reason. This restriction is essential because LLMs are fluent in financial language but must not be permitted to invent unimplemented features or reinterpret metrics after results are observed. The formal layer specifies feature construction, backtesting, performance statistics, and portfolio formation.

4.1. Feature Representation

For each stock i and date t , the feature vector $\mathbf{x}_{i,t}$ is computed from raw prices $p_{i,t}$ and dollar volumes $d_{i,t}$ using only information available at or before date t :

$$\text{mom}_{20}(i, t) = \frac{p_{i,t}}{p_{i,t-20}} - 1 \quad (1)$$

$$\text{mom}_{60}(i, t) = \frac{p_{i,t}}{p_{i,t-60}} - 1 \quad (2)$$

$$\text{mom}_{12-1}(i, t) = \frac{p_{i,t-21}}{p_{i,t-252}} - 1 \quad (3)$$

$$\sigma_{20}(i, t) = \sqrt{252} \hat{\sigma}(\{r_{i,s}\}_{s=t-19}^t) \quad (4)$$

$$\ell(i, t) = \log(1 + \bar{d}_{i,t-19:t}) . \quad (5)$$

where $r_{i,t} = p_{i,t}/p_{i,t-1} - 1$ is the daily return and $\hat{\sigma}(\cdot)$ denotes the sample standard deviation. The raw features are ranked cross-sectionally within each date:

$$\tilde{x}_{i,t}^{(k)} = \frac{\text{rank}_t(x_{i,t}^{(k)}) - 1}{N_t - 1}, \quad N_t > 1. \quad (6)$$

Ranking has two advantages. First, it makes feature thresholds comparable across stocks with different price levels, volatility scales, and liquidity profiles. Second, it constrains the feature space available to the LLM. A condition such as $\tilde{x} > 0.6$ has a stable cross-sectional interpretation, whereas a raw threshold on price, volume, or return may have no stable economic interpretation across securities.

4.2. Backtesting Engine

At each rebalance date τ , hypothesis h defines an indicator $\mathbf{1}_{i,\tau}^{(h)}$ equal to one when stock i satisfies the generated buy condition. Equal weights are assigned over the selected set $S_\tau^{(h)}$:

$$w_{i,\tau}^{(h)} = \begin{cases} 1/|S_\tau^{(h)}|, & i \in S_\tau^{(h)}, \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

With weights carried between rebalance dates, the net portfolio return subtracts transaction costs from turnover:

$$R_t^{(h)} = \sum_i w_{i,t-1}^{(h)} r_{i,t} - c \sum_i |w_{i,t}^{(h)} - w_{i,t-1}^{(h)}|, \quad (8)$$

where $c = 10$ basis points in the demonstration system and turnover is evaluated at rebalance dates. The cost model is intentionally simple, but its inclusion is conceptually important. A signal with attractive gross returns but excessive turnover is not economically equivalent to a signal with the same net return and lower trading intensity.

4.3. Performance Statistics

For a daily excess-return series $\{R_t\}_{t=1}^T$, annualized return, annualized volatility, Sharpe ratio, and maximum drawdown are computed as

$$\mu_{\text{ann}} = 252 \bar{R}, \quad (9)$$

$$\sigma_{\text{ann}} = \sqrt{252} \hat{\sigma}(R), \quad (10)$$

$$\text{Sharpe} = \frac{\mu_{\text{ann}}}{\sigma_{\text{ann}}}, \quad (11)$$

$$\text{MDD} = \min_t \left(\frac{\text{NAV}_t}{\max_{s \leq t} \text{NAV}_s} - 1 \right). \quad (12)$$

These statistics are interpreted jointly. In the demonstration, the risk-free rate is set to zero; a production study would compute excess returns against an appropriate funding benchmark. A high annualized return is not sufficient if drawdown is unacceptable or if volatility estimates are unstable. Similarly, a Sharpe ratio is not reported when the denominator is non-finite or negligible. The objective is to summarize risk-adjusted performance without concealing path dependence.

4.4. Portfolio Construction

Portfolio construction converts a research signal into constrained exposure. Let $a_{i,t}$ denote the composite alpha score and let $\hat{\sigma}_{p,t}$ be the recent realized volatility of the candidate portfolio. The target weight vector is capped and scaled as

$$\tilde{w}_{i,t} = \min \left(0.05, \frac{a_{i,t}^+}{\sum_j a_{j,t}^+} \right), \quad w_{i,t} = \tilde{w}_{i,t} \cdot \min \left(1, \frac{\sigma^*}{\hat{\sigma}_{p,t}} \right), \quad (13)$$

where σ^* is the volatility target and $a^+ = \max(a, 0)$. If all positive alpha scores are zero, the portfolio remains in cash for that rebalance period. The cap prevents a single extreme-ranked name from dominating the portfolio, while the volatility scaling step prevents the system from interpreting alpha evidence as permission for uncontrolled risk exposure.

5. The EvaluationAgent Decision Protocol

The decision protocol formalizes the research continuation problem. In traditional research workflows, this decision is often made informally after inspecting a backtest. That practice is vulnerable to confirmation bias: researchers may continue searching when a favored idea fails, stop too quickly when a result is statistically favorable but unreliable, or forget prior negative evidence. The `EvaluationAgent` makes the decision explicit.

5.1. Motivation

The motivation for a separate governor is methodological discipline and independent control. LLMs can generate hypotheses and summarize failure context, but they should not control the entire research state without structure. The agent separates exploration from evaluation. It allows creative proposal formation while enforcing deterministic checks, round limits, and persistent failure memory. This separation is particularly important in finance, where repeated search over noisy data can produce statistically attractive but spurious strategies [7].

5.2. Two-Step Decision Protocol

The protocol has two stages. The first is a deterministic prescreen. If the cycle is fragile, if the maximum round limit has been reached, or if a kill switch has fired, the decision is made without relying on an additional model call. The second stage is a constrained LLM decision used when judgment is required. The response schema is intentionally small:

```
{
  "decision": "continue | stop_success | halt_max_rounds",
  "reasoning": "short explanation grounded in metrics",
  "failure_summary": "context to inject into the next cycle"
}
```

The schema prevents the model from changing the software contract. It can reason about evidence, but the surrounding code controls what fields are accepted and how those fields are used. This design balances flexibility and control: the LLM contributes semantic interpretation, while deterministic code preserves procedural integrity.

5.3. State Accumulation

State accumulation is handled by the outer agent, not by the inner loop. The agent stores concise failure context, rejected strategy names, and decision logs. This design allows the next cycle to learn from prior evidence while keeping each AI4SLoop instance reproducible. The distinction matters because hidden state is a major source of audit failure. If a later cycle behaves differently, reviewers must be able to determine whether the difference came from new evidence, a changed prompt, or an unrecorded implementation change.

6. Numerical Safeguards

Numerical safeguards are necessary because automated research loops can amplify artifacts. A single error in feature timing, return calculation, or denominator handling can propagate through hypothesis generation and be misrepresented as a coherent financial result. The system therefore imposes safeguards at four levels: data construction, feature calculation, backtesting, and deployment simulation.

At the data level, prices are kept finite and bounded so that synthetic paths do not produce impossible returns. At the feature level, rolling calculations are aligned so that future information is not used in past decisions. At the backtesting level, turnover costs are subtracted explicitly and empty or degenerate portfolios are treated as uninformative. At the deployment level, volatility scaling and kill-switch logic prevent a simulated strategy from continuing through unacceptable losses.

The safeguards also address data-snooping risk. Multiple hypothesis generation is unavoidable in an AI-assisted system; indeed, it is one of the reasons to use such a system. But repeated search must be paired with stronger rejection criteria. A strategy that fails under a liquidity crisis or crowding decay scenario should not be rescued by a favorable base-case narrative. Similarly, statistics with unstable denominators should not be interpreted as evidence of predictive content or economic value.

These controls are deliberately conservative. They reduce the probability of false positives at the cost of rejecting some candidates that might have been profitable under more favorable assumptions. For a research system intended to withstand external scrutiny, that trade-off is appropriate. The objective is not to maximize the number of attractive backtests; it is to maximize the credibility of the surviving hypotheses.

7. LLM Integration

The LLM is integrated as a constrained reasoning component. It is not treated as an oracle and does not directly control portfolio weights or risk limits. Its main functions are hypothesis generation, stress interpretation, performance feedback, update generation, and agent decision support. Each function is governed by a prompt contract and a parser.

The implementation uses five prompt families:

1. **Hypothesis generation** — returns named alpha hypotheses, each with an economic rationale and a parseable `buy_condition`.
2. **Stress interpretation** — receives the scenario table and returns a concise deployment recommendation.
3. **Performance feedback** — compares expected and realized metrics and returns a diagnosis token such as `SIGNAL_INTACT`, `SIGNAL_DEGRADED`, `REGIME_CHANGE`, or `OVERFITTED`.
4. **Hypothesis update** — returns an improved current hypothesis and two search seeds for the next cycle.
5. **Agent decision** — returns the decision, reasoning, and failure summary required by the outer governor.

All prompts instruct the model to return structured JSON rather than Markdown or explanatory prose. A parsing utility strips accidental formatting before calling `json.loads()`. Malformed outputs are not treated as evidence; they are treated as interface failures. This distinction is essential in a financial setting. A narrative explanation is not a valid trading rule, and an unparseable recommendation is not a governance decision.

The model can be replaced without changing the architecture. The relevant object of governance is not the identity of a particular provider, but the contract between language reasoning and deterministic evaluation. Stronger models may improve the quality of hypotheses and diagnoses, but they do not remove the need for feature restrictions, cost-aware tests, stress scenarios, and audit trails.

8. Stress Testing Scenarios

Stress testing asks whether a candidate signal remains credible when key market assumptions are challenged. The scenarios are not intended to span all possible market states; they are designed to test three common sources of fragility in systematic equity strategies: volatility sensitivity, liquidity reliance, and alpha crowding.

Scenario A — Volatility Spike. A multiplicative perturbation $p_{i,t}^{(A)} = p_{i,t}(1 + \epsilon_{i,t})$, with $\epsilon_{i,t} \sim \mathcal{N}(0, 0.005^2)$, is applied to daily prices, and perturbed prices are clipped to [85%, 115%] of their base values. This scenario tests whether the signal depends excessively on a stable ranking structure that may deteriorate when idiosyncratic volatility rises.

Scenario B — Liquidity Crisis. Stocks in the top 20% of the cross-sectional volatility distribution are removed from the investable universe at each rebalance date. This scenario is material because many backtests implicitly assume that the most statistically interesting names are tradable at scale. In stressed markets, that assumption is often false.

Scenario C — Crowding Decay. All daily returns are scaled to 70% of their base values: $r_{i,t}^{(C)} = 0.70 \cdot r_{i,t}$. This scenario approximates alpha decay when a signal becomes crowded or when competition arbitrages away the edge. The mechanism is simplified, but the conceptual test is material: a strategy whose economics depend on large unattenuated returns may not be robust.

A strategy is classified as fragile if any scenario produces a Sharpe ratio below zero or a maximum drawdown worse than -40% . This rule is intentionally strict. A strategy that fails one adverse scenario may still contain useful research information, but it should not proceed as if the failure were irrelevant. The failure summary becomes part of the next research prompt, allowing the system to search for hypotheses that address the observed weakness.

9. Discussion and Limitations

The architecture should be evaluated as a research-governance design. Its principal strength is the explicit connection between generative AI, deterministic financial testing, stress analysis, and agentic decision logging. Its principal limitation is that the empirical environment remains simplified. The appropriate question, therefore, is not whether the demonstration proves a deployable strategy; it is whether the architecture enforces a more rigorous standard for AI-assisted alpha research.

9.1. Contributions

The primary contribution is a formal mapping from AI4S to quantitative finance. The paper specifies how observation, representation, generation, validation, feedback, and update can be implemented as software components with typed interfaces. A second contribution is the separation between `AI4SLoop` and `EvaluationAgent`. This separation prevents the component that generates and tests hypotheses from also owning the multi-round governance state. A third contribution is the emphasis on stress-triggered loop-back: weak results are not only reported, but converted into structured context for subsequent search.

9.2. Limitations

The most material limitation is the use of synthetic data. Real-world equity research requires survivorship-bias-free universes, corporate-action handling, point-in-time fundamentals, realistic borrow assumptions, and reliable execution data. The current transaction-cost model is also simplified. Real costs depend on spread, market impact, participation rate, queue position, and position size; a production system would require a more sophisticated execution model, such as the nonlinear frameworks associated with optimal execution research [6].

A second limitation is LLM stochasticity. Different model calls may produce different hypotheses under the same prompt. This is not necessarily a defect; diversity can be useful in hypothesis search. However, it requires rigorous logging, multiple seeds, and out-of-sample evaluation. A third limitation is the restricted signal language. The current buy conditions operate over a small feature vocabulary. Richer economic hypotheses involving sector neutrality, interaction terms, factor residualization, or event timing would require a more expressive specification language.

9.3. Future Work

Future work should proceed along four lines. First, synthetic data should be replaced with point-in-time market and fundamental data. Second, the feature space should be broadened to include earnings revisions, options-implied volatility, balance-sheet quality, short interest, and order-flow measures. Third, the portfolio layer should incorporate factor constraints, capacity analysis, and nonlinear transaction-cost estimation. Fourth, the agent layer should be extended to compare multiple parallel research loops and escalate borderline decisions to human review.

A particularly material extension is statistical control for repeated search. An AI-assisted system can generate many plausible hypotheses quickly, which increases the need for holdout samples, cross-regime

validation, and explicit control of backtest overfitting [7]. The long-run research agenda is therefore not simply to make the LLM more creative. It is to make the entire discovery process more falsifiable.

10. Conclusion

This paper has presented a closed-loop architecture for applying AI4S principles to algorithmic stock selection. The architecture combines LLM-generated hypotheses with deterministic feature construction, cost-aware backtesting, scenario stress testing, constrained portfolio formation, and an outer decision agent. Its central design principle is that model creativity must be paired with empirical discipline. The LLM expands the hypothesis space; the financial tests discipline and restrict it.

The proposed system is not a claim of deployable trading profitability. It is a method for organizing AI-assisted research so that hypotheses, metrics, failures, and decisions remain auditable. That distinction is essential. Financial markets reward robust evidence and penalize overconfident narratives. A credible AI trading research system must therefore preserve uncertainty, record negative evidence, penalize fragility, and prevent the model from changing the rules after observing results.

The broader implication is that AI4S can contribute to quantitative finance when it is implemented as a governed discovery process. Observation, representation, generation, validation, stress testing, feedback, and revision are not metaphors; they are software interfaces and statistical controls. The companion notebooks make this claim operational: the first implements the architecture faithfully, and the second extends it into a governed, audit-oriented research system with additional loops and formal review artifacts. The architecture presented here provides one concrete blueprint for such a system. Its value lies not in automating judgment away, but in making judgment more structured, more transparent, and more accountable.

Appendix A. Companion Computational Notebooks

The paper is accompanied by two computational notebooks that serve different roles in the research record. The first notebook, available as [Companion Notebook I](#), is a faithful computational implementation of the architecture described in this paper. It preserves the core AI4S structure: an `AI4SLoop` class encapsulates one complete research cycle, while an `EvaluationAgent` governs continuation, stopping, and failure-context injection across rounds. The notebook includes synthetic data generation, LLM-generated hypotheses, backtesting, stress testing, paper trading, deployment simulation, feedback, and a multi-round diagnostic dashboard. Its purpose is to make the conceptual architecture executable, inspectable, and reproducible.

The second notebook, available as [Companion Notebook II](#), extends the same core design into a more institutional research system. It adds a governance-oriented layer of controls and audits rather than repeating the proof-of-concept demonstration. The most consequential change is that executable LLM-generated `buy_condition` strings are replaced by structured JSON hypotheses based on approved feature weights. This reduces the risk of directly evaluating model-generated trading logic and makes the strategy layer easier to parse, validate, repair, and audit.

The governed notebook also deepens the closed-loop process. It requires at least five research cycles and permits up to ten, rather than stopping at a small number of rounds. This forces additional loops of failure-context injection, model critique, and hypothesis repair, while still imposing a hard boundary against uncontrolled search and overfitting. The objective is not to reward indefinite optimization; it is to give the system enough opportunity to learn from structured failures before a final governance decision is made.

Finally, the second notebook adds an explicit institutional audit layer. Its artifacts include a model card, risk register, approval checklist, reproducibility manifest, cryptographic hashes, model-gate decision, plain-language critique, final research memo, and exported audit bundle. The distinction between the two notebooks is therefore substantive. The first notebook shows the AI4S trading loop operating as a compact proof-of-concept demonstration. The second notebook preserves that loop but adds structured-output discipline, deterministic validation, stronger governance, explicit non-deployment boundaries, and reviewable artifacts suitable for an institutional research-control environment.

Appendix B. Workflow Diagram

Figure B.1 provides the workflow diagram corresponding to the closed-loop AI4S trading procedure described in Section 2. It summarizes the movement from observations and feature construction to hypothesis generation, empirical testing, stress analysis, governance review, and loop-back revision.

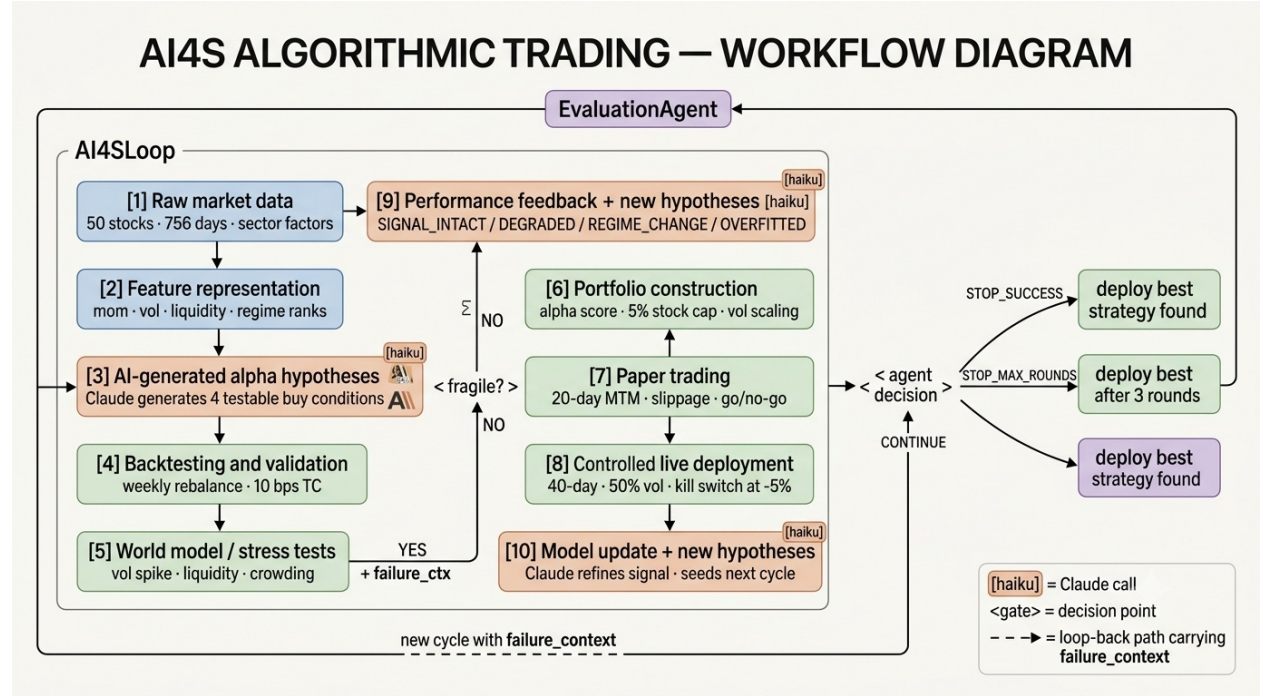


Figure B.1: Workflow diagram for the agent-governed AI4S trading architecture.

References

- [1] H. Wang *et al.*, “Scientific discovery in the age of artificial intelligence,” *Nature*, vol. 620, pp. 47–60, 2023.
- [2] J. Jumper *et al.*, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, pp. 583–589, 2021.
- [3] K. T. Butler, D. W. Davies, H. Cartwright, O. Isayev, and A. Walsh, “Machine learning for molecular and materials science,” *Nature*, vol. 559, pp. 547–555, 2018.
- [4] B. Burger *et al.*, “A mobile robotic chemist,” *Nature*, vol. 583, pp. 237–241, 2020.
- [5] M. López de Prado, *Advances in Financial Machine Learning*. Wiley, 2018.
- [6] R. Almgren and N. Chriss, “Optimal execution of portfolio transactions,” *Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2001.
- [7] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu, “The probability of backtest overfitting,” *Journal of Computational Finance*, vol. 20, no. 4, pp. 39–69, 2017.
- [8] J. Y. Campbell, A. W. Lo, and A. C. MacKinlay, *The Econometrics of Financial Markets*. Princeton University Press, 1997.